AI Assignment Solutions

Question 1: Dry Run of A* Search Algorithm

Graph edges (weights) used from the diagram:
A–B = 2, A–C = 4, B–C = 3, B–D = 7, B–E = 2, C–E = 3, D–E = 2, D–G = 2

Heuristic values h(n): A=7, B=6, C=4, D=3, E=2, G=0 *(Nodes H and the disconnected right-hand component are not part of the A→G path and are omitted from the search.)*

Step-by-step Iterations

Iter	Current Node	g(n)	h(n)	f(n)=g+h	Open List (node: f)	Closed List
0	—	—	—	—	A:7	—
1	**A**	0	7	7	B:8, C:8	A
2	**B**	2	6	8	C:8, D:12, E:6	A, B
3	**E**	4	2	6	C:8, D:9	A, B, E
4	**C**	4	4	8	D:9	A, B, E, C
5	**D**	6	3	9	G:8	A, B, E, C, D
6	**G** (Goal!)	8	0	8	—	A, B, E, C, D, G

Explanation of key steps:

- **Iteration 1:** Expand A → generate B (g=2,h=6,f=8) and C (g=4,h=4,f=8).
- **Iteration 2:** B and C tie at f=8; B is expanded first. Generates C via B (g=5,f=9) — worse than existing C (f=8), discarded. Generates D (g=9,f=12) and E (g=4,f=6).
- **Iteration 3:** Lowest f is E (f=6). Expanding E finds a cheaper route to D: g=6, f=9 (replaces the old D f=12). Route to C via E (f=11) is worse than existing, discarded.
- **Iteration 4:** Lowest f is C (f=8). All neighbours of C (A, B, E) are already closed, so nothing new is added.
- **Iteration 5:** Lowest f is D (f=9). Expanding D generates G: g=8, h=0, f=8.
- **Iteration 6:** G is selected from the open list (only node remaining) and is the goal → search terminates.

1. Optimal Path: A → B → E → D → G `(2 + 2 + 2 + 2)`

2. Total Path Cost: 2 + 2 + 2 + 2 = **8**
(Matches g(G) = 8, confirming optimality.)

Question 2: Dry Run of Minimax with Alpha–Beta Pruning

Tree structure:
MAX (root)
  ├── MIN1 → leaves: 3, 5, 6
  └── MIN2 → leaves: 9, 1, 2

Step-by-step Evaluation (left to right)

Node Visited	α (before/after)	β (before/after)	Value / Action
Leaf 3 (under MIN1)	−∞	∞ → 3	β = min(∞,3) = 3
Leaf 5 (under MIN1)	−∞	3 → 3	β = min(3,5) = 3 (no change)
Leaf 6 (under MIN1)	−∞	3 → 3	β = min(3,6) = 3 (no change)
MIN1 result	−∞	3	**MIN1 = min(3,5,6) = 3**
MAX (after MIN1)	−∞ → 3	∞	α = max(−∞,3) = 3
Leaf 9 (under MIN2)	3	∞ → 9	β = min(∞,9) = 9; α(3) < β(9), continue
Leaf 1 (under MIN2)	3	9 → 1	β = min(9,1) = 1; now α(3) ≥ β(1) → **PRUNE**
Leaf 2 (under MIN2)	—	—	**PRUNED – never visited (α–β cutoff)**
MIN2 result	3	1	**MIN2 = 1** (search cut off, remaining child cannot lower it further in a way that matters)
MAX (root) result	3	∞	**MAX = max(3, 1) = 3**

Why leaf "2" is pruned: After seeing leaf 1 under MIN2, β drops to 1. Since α = 3 (the best value MAX can already guarantee via MIN1) is ≥ β = 1, MAX would never let the game proceed into MIN2's branch (MIN2 can only return a value ≤ 1, which is worse for MAX than 3). Hence the third child (value 2) of MIN2 is never evaluated.

1. Best Move for MAX: Choose the **left branch (MIN1)**, which yields value 3.

2. Final Minimax Value: 3

3. Pruned Nodes: Leaf node with value **2** (the third/rightmost child of MIN2) is pruned.

Prepared as a dry-run solution based on the submitted graph and game-tree diagrams.